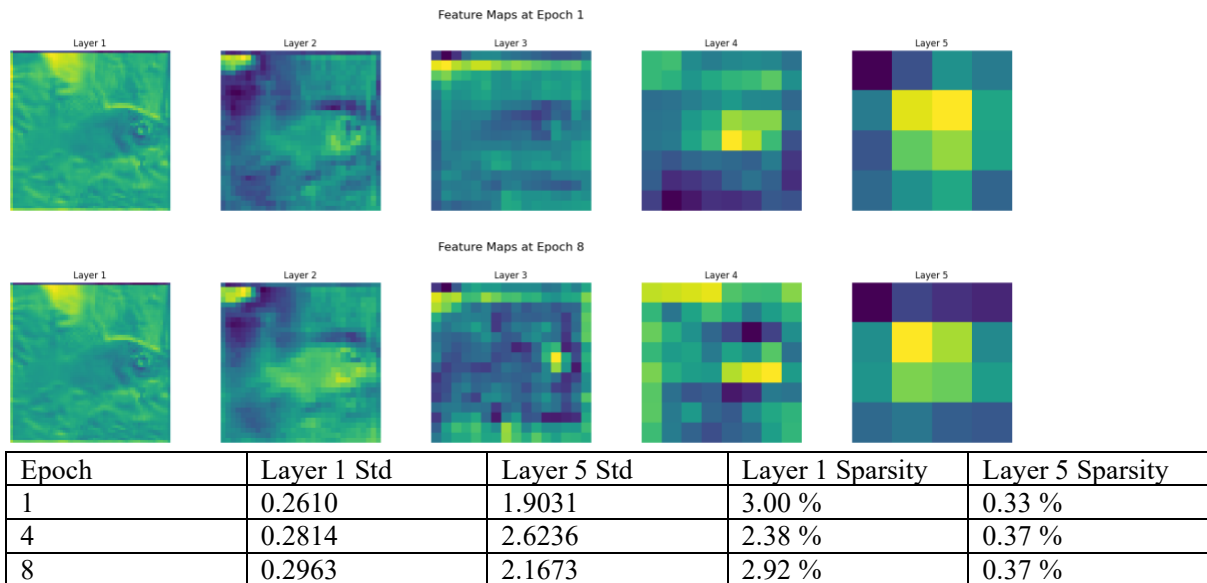


Assignment4_20201118_Analysis

1. CNN

T3. Analyze the feature maps (visualization) stored per epoch and evaluate how the learned representations change throughout training, focusing on the differences between the first layer and the last layer.

This section examines how the CNN's feature maps change during training, focusing on the first and last convolutional layers. Feature maps were saved for every epoch, and both visual patterns and activation statistics were compared.



Differences Between Layers

- Layer 1

Shows fine-grained, texture-like patterns that remain visually similar from Epoch 1 to Epoch 8.

Activation variance increases slightly across epochs (std 0.26 $\rightarrow$ 0.29).

- Layer 5

Shows low-resolution, spatially selective responses.

Activation variance is higher than in Layer 1 (std 1.9–2.6) and sparsity remains very low (<0.4%).

Changes Across Epochs

- Layer 1 shows a mild upward change in variance.
- Layer 5 shows substantial variance changes, peaking at Epoch 4.
- Depth-related differences persist, but Layer 5 also exhibits notable epoch-wise variation.

Summary

Layer 1 remains visually consistent with small increases in variance, while Layer 5 shows higher variance, lower sparsity, and a non-linear variance change across epochs.

Q1. (Extra credit) If you change the hyperparameters, how do the feature maps change?

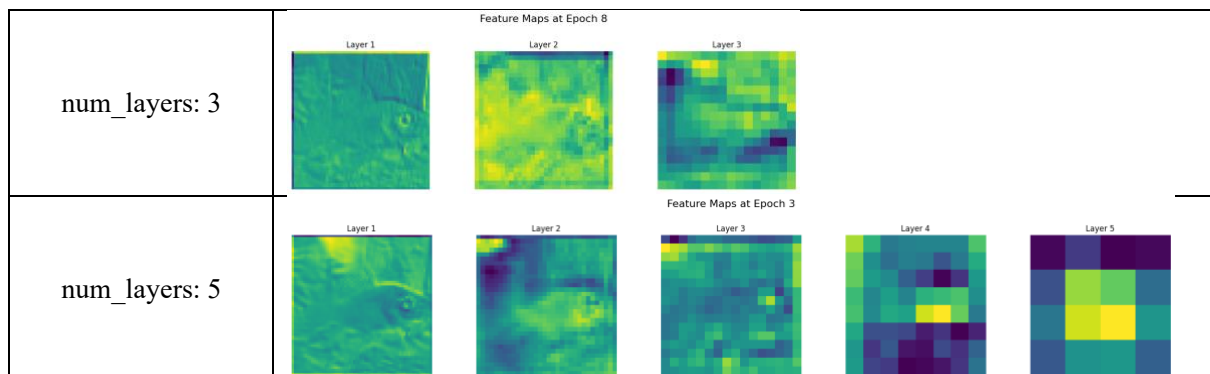
This section compares feature maps produced by different CNN hyperparameter settings.

(1) number of layers (3 vs. 5)

(2) base number of filters (16 vs. 32)

Only the final-epoch feature maps were analyzed, since intermediate epochs showed only minor differences.

(1) num_layers



- Spatial resolution of the deepest layer

The 5-layer model produces a much lower-resolution final feature map, because more pooling operations reduce the spatial size.

The deepest feature map in the 5-layer network contains fewer spatial patches than in the 3-layer network.

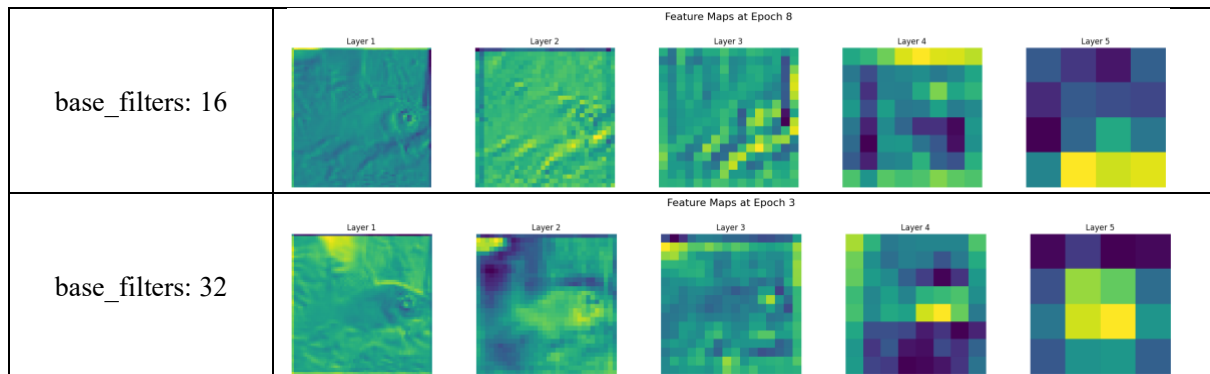
- Strength of early-layer activations

The first-layer feature map in the 3-layer model appears weaker and lower in contrast.

The 5-layer model shows stronger and more pronounced edge/texture responses in the first layer.

With more layers, earlier filters become more expressive under the same training conditions.

(2) base_filters



- Early-layer contrast

The model with 32 base filters shows stronger and clearer edge/texture activations in Layer 1.

The 16-filter model appears more muted.

Increasing the number of filters expands representational capacity and results in stronger channel responses.

- Deeper-layer sharpness

Deeper layers (Layer 3–5) exhibit slightly sharper activation regions when base_filters = 32, although the overall structure remains similar.

More filters allow the model to learn a broader variety of feature channels, which produces slightly more distinct activation patterns.

Summary

Changing num_layers affects the spatial size of deeper feature maps and the contrast of early-layer activations.

Changing base_filters affects the strength of activations rather than their spatial structure.

2. RNNs

T3. Try adjusting the epochs and learning rate, and analyze which model shows the best performance.

This experiment compares three models (Elman RNN, Jordan RNN, LSTM) on the sunspots prediction task by sweeping the learning rate and the number of epochs. Mean squared error (MSE) on the test set is used as the evaluation metric.

Learning rate sweep (epochs fixed at 25)

Learning rates {0.1, 0.01, 0.001, 0.0001} were tested while keeping the number of epochs fixed to 25.

	LR = 0.1	LR = 0.01	LR = 0.001	LR = 0.0001
ElmanRNN	1625.67	2098.52	5064.66	7443.06
JordanRNN	1462.08	2850.93	5064.90	7472.14
LSTM	3860.46	3018.86	4978.85	7525.46

Across all models, LR = 0.1 gives the lowest MSE.

At this learning rate, Jordan RNN achieves the best performance (MSE $\approx$ 1462), followed by Elman RNN and LSTM.

Epoch sweep (learning rate fixed at 1e-2)

Next, the number of epochs was varied while fixing the learning rate to 0.01.

	EPOCHS = 10	EPOCHS = 25	EPOCHS = 100	EPOCHS = 1000
ElmanRNN	3358.02	2551.13	773.69	556.80
JordanRNN	3578.43	3495.43	1709.18	1045.64
LSTM	3502.42	2556.02	502.51	480.79

Increasing the number of epochs improves performance for Elman RNN and LSTM.

At 100 epochs, LSTM gives the lowest MSE among the three models.

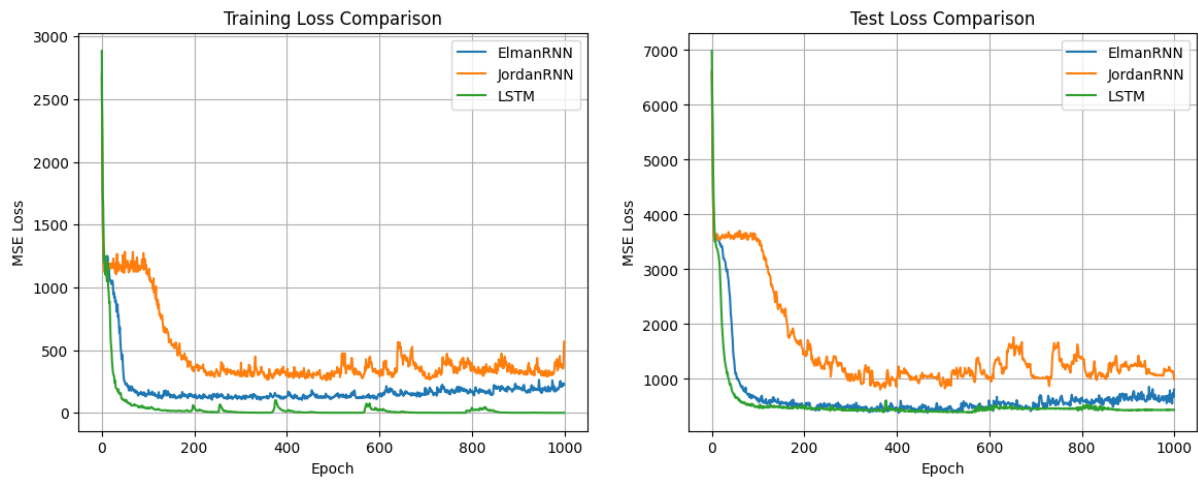
At 1000 epochs, LSTM achieves the lowest MSE ($\approx$ 481), outperforming Elman RNN ($\approx$ 557).

Jordan RNN improves steadily as the number of epochs increases, but its MSE remains higher than the other two models across all settings.

Best model summary

- With small training budgets (10–25 epochs), Elman RNN performs best, while LSTM performs best at 100 and 1000 epochs.
- Jordan RNN performs worse than the other two in most settings.

Q1. Which model generally performs better, and why?



Across all experiments, the LSTM model shows the best overall performance.

Although the Elman RNN performs slightly better when the training budget is extremely small, the LSTM quickly surpasses it as training continues and ultimately achieves the lowest test loss among the three architectures. Jordan RNN improves with more epochs but consistently remains the worst-performing model.

The loss curves provide the clearest explanation for this behavior: after the initial transient phase, LSTM's training and test losses decrease more steadily and settle at a lower level than those of Elman and Jordan. This indicates that LSTM's gating mechanism enables more stable learning and allows it to capture longer temporal dependencies in the sunspot sequence, whereas Elman and Jordan rely solely on simple recurrent connections that are more sensitive to vanishing gradients and optimization instability.

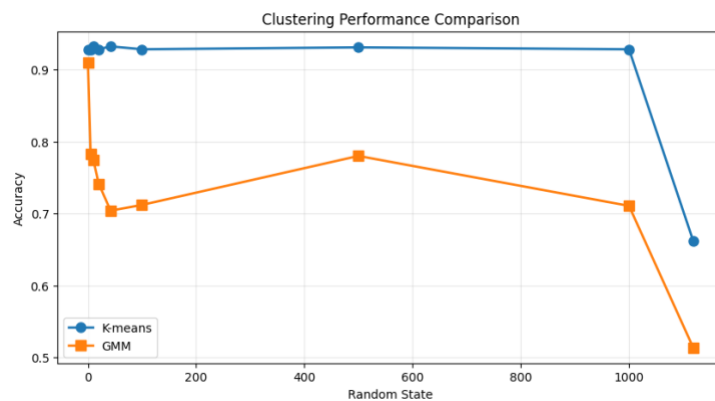
Overall, the LSTM is the model that generally performs best because it combines stable optimization with effective modeling of long-range patterns, while Elman performs reasonably well under small training budgets and Jordan remains the least reliable across most settings.

3. K-Means Clustering and GMM

T3. Compare the clustering results of K-means and GMM, and analyze which model performs better. Discuss the reasons behind the performance differences (Try changing the random state).

We compare the clustering performance of K-means and GMM under various random states. Both methods use the same number of clusters, and accuracy is computed by finding the best permutation between predicted cluster labels and ground-truth classes. The experiment was repeated across nine random states.

Overall performance



Across most random states, K-means consistently achieves higher clustering accuracy than GMM.

The mean and standard deviation of the accuracies across all runs are:

- K-means: Mean = 89.96%, Std = 8.42%
- GMM: Mean: 73.64%, Std: 9.86%

This shows that K-means not only performs better on average, but also exhibits lower variability across random initializations.

Behavior across random states

- For random states 0, K-means and GMM accuracy are 92.80%, 90.93%
- For random states 5, 10, 20, 42, 100, 500, and 1000, K-means maintains accuracy around 92–93%, while GMM fluctuates between 70–80%.
- Both models drop sharply at random state 1119 (K-means: 66.13%, GMM: 51.33%), but the gap between them remains similar.
- GMM is noticeably more sensitive to initialization: its accuracy varies by more than 20 percentage points across runs, while K-means only shows a large drop in one outlier case.

Which model performs better?

Based on the results, K-means performs better in this dataset.

It achieves higher accuracy in nearly all random states and is more stable across different initializations.

T4. (Extra credit) If possible, extract features from a CNN, apply PCA for dimensionality reduction, and replace the classifier with a clustering model. Compare the performance of K-means and GMM on the reduced feature space and interpret the findings (v.s. CNN Classifier).

We extract features from the CNN's penultimate layer on the validation set (10,000 samples, 2048-D), then apply PCA to reduce the dimensionality to 50. On this reduced feature space we replace the classifier with unsupervised clustering models: K-means and a Gaussian Mixture Model (GMM) with the same number of clusters as classes (200).

Clustering results

K-means and GMM are evaluated using cluster purity with respect to the ground-truth labels.

- K-means purity: 17.91%
- GMM purity: 2.49% (GMM converged after 1 EM iteration)

Even though both numbers are much lower than the supervised CNN classifier accuracy, K-means reaches a clearly higher purity than GMM in this setting.

Visualization of the feature space

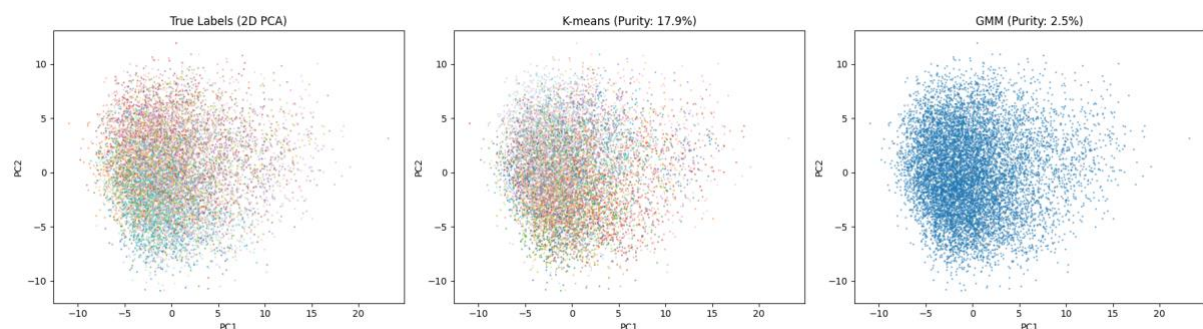


Figure shows a 2D PCA projection of the CNN features, colored by true labels (left), K-means clusters (middle), and GMM clusters (right). In the true-label plot, most points form one large, highly overlapping cloud, and clear class-specific groups are hard to see. The K-means assignments still look mixed but show some local structure, which is consistent with its moderate purity (~18%). In contrast, the GMM plot looks almost uniform, suggesting that many samples are assigned to similar components, in line with the very low purity (~2.5%).

Interpretation vs CNN classifier

These observations suggest that, after PCA, the CNN feature space does not form well-separated clusters that align with the 200 classes, so unsupervised clustering cannot match the supervised CNN classifier. K-means performs relatively better, possibly because it only relies on distances, whereas GMM needs to estimate high-dimensional covariance matrices from limited data and appears sensitive to this, converging quickly with poor separation. Overall, the experiment indicates that the CNN learns useful but entangled representations that benefit strongly from supervised training rather than purely unsupervised clustering.